Università degli Studi di Messina

CdL in Scienze Informatiche
Sistemi Operativi - Modulo B
A.A. 2024/2025

Progetto di un Mini-Laboratorio Virtualizzato e Automatizzato

Sandi Russo
553675

26 gennaio 2026

Sommario

La presente relazione ha lo scopo di illustrare il processo di progettazione e implementazione di un laboratorio virtualizzato multi-macchina. Utilizzando le linee guida fornite dal docente, è stata creata un'architettura web a tre livelli, completamente automatizzata e resiliente, composta da quattro macchine virtuali: un bilanciatore di carico (load balancer), due server web e un server di database. L'intera infrastruttura è stata codificata utilizzando **Vagrant**, seguendo il principio dell'*Infrastructure as Code* (IaC), mentre la configurazione del software su ciascuna macchina è stata realizzata in modo dichiarativo attraverso **Ansible**, che si occupa della gestione della configurazione e dell'installazione del software nelle macchine virtuali. Inoltre, il progetto prevede procedure automatizzate per la manutenzione, che comprendono la gestione di snapshot per ripristini rapidi e la realizzazione di backup completi in formato OVA per il disaster recovery. La validazione conclusiva attesta non soltanto il corretto funzionamento dell'intera architettura, ma anche la sua capacità di resistere a malfunzionamenti simulati, garantendo così un servizio di elevata affidabilità.

Indice

1 Introduzione **1**
1.1 Concetti Teorici . . . 1
1.1.1 Virtualizzazione e Hypervisor . . . 1
1.1.2 Infrastructure as Code (IaC) . . . 2
1.2 Architettura a tre livelli . . . 2

2 Definizione dell'infrastruttura: Vagrantfile **4**
2.1 Analisi del Vagrantfile . . . 4
2.1.1 Blocco 1: definizione dei dati di configurazione . . . 4
2.1.2 Blocco 2: ciclo di creazione e configurazione di base . . . 5
2.1.3 Blocco 3: configurazione del provider e provisioning . . . 5

3 Gestione della configurazione con Ansible **7**
3.1 Struttura e principi dei Playbook . . . 7
3.1.1 Struttura di un Playbook . . . 7
3.2 Playbook comune . . . 8
3.3 Playbook del Load Balancer . . . 8
3.4 Playbook dei Web Server . . . 9
3.5 Configurazione del Reverse Proxy (Load Balancer) . . . 9
3.6 Playbook del Database . . . 10
3.7 Gestione dei dati sensibili: Ansible Vault . . . 10

4 Procedure di gestione e manutenzione **11**
4.1 Gestione degli snapshot . . . 11
4.1.1 Snapshot vs. backup . . . 11
4.2 Backup completo e migrazione (Disaster Recovery) . . . 12
4.2.1 Blocco 1: inizializzazione . . . 12
4.2.2 Blocco 2: gestione dello stato della VM . . . 12
4.2.3 Blocco 3: esportazione, riavvio e pulizia . . . 13

5 Verifica funzionale e test di resilienza **15**
5.1 Test del Load Balancing . . . 15
5.2 Test di alta affidabilità (Failover) . . . 16
5.2.1 Test di ripristino (Recovery) . . . 17

6 Conclusioni e sviluppi futuri **18**
6.1 Possibili Sviluppi Futuri . . . 18

1 Introduzione

Il progetto nasce dall'esigenza di creare un ambiente di laboratorio virtuale che non fosse una semplice aggregazione di macchine virtuali, ma un sistema complesso, realistico e gestito con metodologie moderne. L'obiettivo è stato quello di ingegnerizzare un'architettura web a tre livelli, capace di replicare scenari reali e di garantire elevata affidabilità, automazione e riproducibilità. Alla base di questo lavoro vi sono i paradigmi contemporanei del DevOps e dell'ingegneria di sistema, che trattano l'infrastruttura non come hardware fisico, ma come software: codificata, versionabile e replicabile. Questa transizione riflette un cambiamento fondamentale nell'amministrazione di sistema, che si sposta da configurazioni manuali, spesso soggette a errori, verso una disciplina ingegneristica dove gli ambienti sono costruiti con lo stesso rigore delle applicazioni software. Lo scopo finale è la creazione di sistemi non solo funzionali, ma intrinsecamente resilienti, scalabili e manutenibili nel tempo.

1.1 Concetti Teorici

1.1.1 Virtualizzazione e Hypervisor

Il pilastro del progetto è la **virtualizzazione**, una tecnologia che consente di creare versioni virtuali di risorse informatiche, come server, storage o reti. Una **Macchina Virtuale (VM)** è un'emulazione software di un computer fisico che astrae le componenti hardware sottostanti. Di conseguenza, il sistema operativo guest interagisce con un set standard di dispositivi emulati (BIOS, dischi, schede di rete), del tutto ignaro del fatto che l'hardware non sia fisico. Le VM utilizzano le risorse della macchina ospitante (host) per eseguire un sistema operativo completo e isolato (guest). L'hypervisor, inoltre, gestisce la creazione di componenti di rete virtuali, come schede di rete (vNIC) per ciascuna VM e switch virtuali, che permettono alle macchine di comunicare tra loro in una rete isolata o di connettersi alla rete fisica esterna. Questo offre vantaggi cruciali:

- **Isolamento:** un problema su una VM (es. un crash del software o una vulnerabilità) non si propaga all'host o alle altre VM, garantendo la stabilità e la sicurezza dell'intero sistema;
- **Efficienza:** più sistemi possono operare simultaneamente su un unico hardware fisico, ottimizzandone l'uso e riducendo i costi energetici e di manutenzione;
- **Portabilità e Riproducibilità:** l'intero stato di una VM, inclusi i dischi virtuali e la configurazione, è incapsulato in un insieme di file. Ciò la rende facilmente trasferibile, permettendo di clonare, spostare o ripristinare un intero ambiente di lavoro con grande semplicità.

Per la gestione delle VM è stato scelto **Oracle VirtualBox**, un *hypervisor* di tipo 2. A differenza degli hypervisor di tipo 1 (bare-metal, es. VMware ESXi) che si installano direttamente sull'hardware, un hypervisor di tipo 2 viene eseguito come un'applicazione sul sistema operativo host (in questo caso, Windows 11), rendendolo ideale e accessibile per ambienti di sviluppo e test su workstation personali.

1.1.2 Infrastructure as Code (IaC)

Il principio dell'**Infrastructure as Code (IaC)** è centrale in questo progetto. Consiste nel gestire e provisionare l'infrastruttura (server, reti, load balancer) attraverso file di configurazione leggibili dall'uomo, anziché tramite configurazione manuale. Questo paradigma permette di trattare l'infrastruttura con le stesse pratiche utilizzate per lo sviluppo software. I principali vantaggi sono:

- **Automazione e velocità:** l'intera infrastruttura può essere creata, aggiornata o distrutta eseguendo uno script, riducendo drasticamente i tempi di setup e deployment.
- **Consistenza:** elimina le discrepanze tra ambienti e previene il "configuration drift" (modifiche manuali non tracciate che rendono i sistemi disallineati).
- **Versionamento:** i file di configurazione possono essere gestiti con sistemi di controllo versione come Git, permettendo di tracciare ogni modifica, collaborare in team e ripristinare versioni precedenti in caso di problemi.

In questo progetto, l'IaC è stata implementata con la combinazione di **Vagrant**, per la creazione delle VM, e **Ansible**, per la gestione dichiarativa della loro configurazione software. È importante sottolineare la sinergia tra questi strumenti: Vagrant agisce come controllore a un livello superiore, occupandosi del ciclo di vita delle macchine (creazione, distruzione, configurazione di rete e risorse). Una volta che la VM è attiva, Vagrant passa il testimone ad Ansible, che opera all'interno del sistema operativo guest per la gestione granulare del software, installando pacchetti e configurando servizi. Questa combinazione permette di separare nettamente la gestione dell'infrastruttura da quella della configurazione, creando un flusso di lavoro pulito e modulare.

1.2 Architettura a tre livelli

È stata progettata un'architettura a tre livelli (3-tier), un modello standard per le applicazioni web che separa la logica in tre strati distinti, ciascuno con responsabilità specifiche. Questa separazione dei compiti (separation of concerns) è fondamentale per costruire sistemi robusti, perché permette a ciascun livello di essere sviluppato, gestito e scalato in modo indipendente. Ad esempio, è possibile aumentare il numero di web server per gestire un picco di traffico senza dover modificare il livello del database. Inoltre, questo design migliora l'isolamento dei guasti (fault isolation): un problema nel livello di presentazione difficilmente si propagherà al livello dati, aumentando la resilienza complessiva. Tale approccio permette anche una gestione delle risorse più efficiente ed economica, scalando selettivamente solo il componente che rappresenta il collo di bottiglia. Questa modularità costituisce anche un pilastro per un design sicuro, limitando la superficie di attacco complessiva del sistema.

- **Presentation Tier (Load Balancer):** un server **Nginx** funge da *reverse proxy* e bilanciatore di carico. È l'unico punto di ingresso (*single point of entry*) per tutto il traffico HTTP. Il suo compito è distribuire le richieste in arrivo tra i server dello strato successivo e mascherare la topologia della rete interna.

Svolge anche un ruolo attivo nel monitoraggio della salute del sistema, eseguendo controlli periodici (health checks) per inoltrare il traffico solo ai web server funzionanti. In scenari di produzione, questo livello è anche il punto ideale per gestire la terminazione SSL/TLS, decifrando il traffico HTTPS e alleggerendo così il carico computazionale dei web server, oltre a poter implementare meccanismi di caching per contenuti statici;

- **Application Tier (Web Servers):** due server web identici (`web1` e `web2`) con Nginx. Ospitano la logica dell'applicazione (che in questo caso è un semplice web server statico, ma in un'applicazione reale conterrebbe il codice sorgente). La loro ridondanza è la chiave per l'**alta affidabilità** (High Availability): se uno dei due server smette di funzionare, l'altro continua a servire le richieste, garantendo la continuità del servizio in una configurazione di tipo attivo-attivo;

- **Data Tier (Database Server):** un server dedicato (`db`) con **MariaDB** per la persistenza dei dati. È isolato e accessibile solo dalla rete privata interna (dai web server), una pratica di sicurezza fondamentale ("defense in depth") per proteggere i dati sensibili da accessi diretti non autorizzati provenienti da Internet.

Tutte le VM sono basate su **Ubuntu Jammy 22.04** e interconnesse tramite una rete privata isolata (`192.168.50.0/24`).

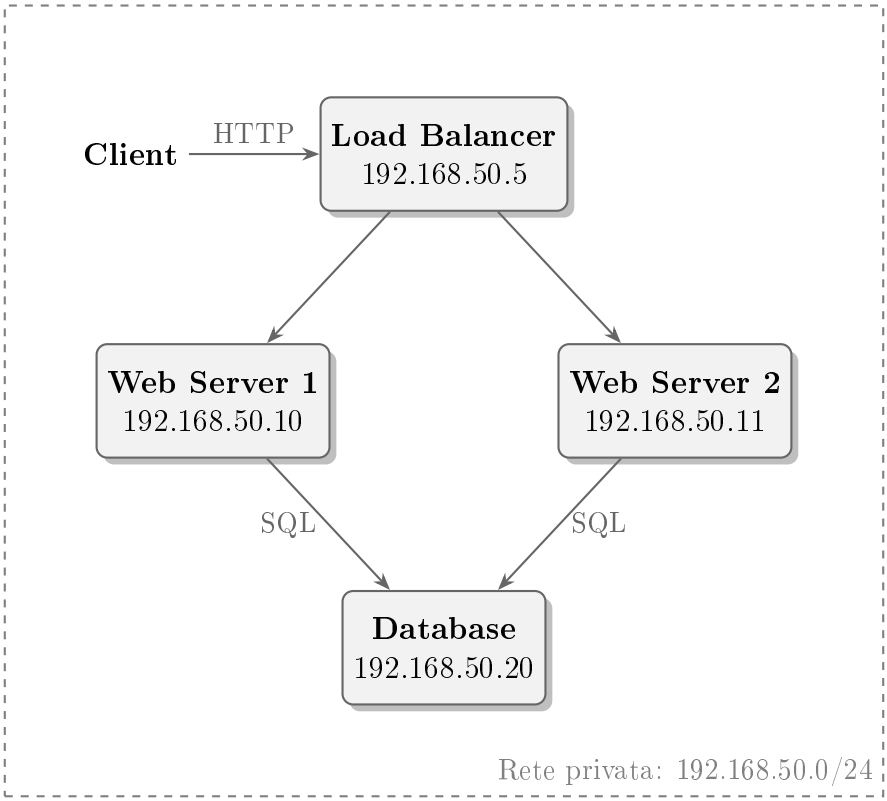

Figura 1: Diagramma dell'architettura a tre livelli del laboratorio

2 Definizione dell'infrastruttura: Vagrantfile

Il cuore dell'approccio IaC è il `Vagrantfile`, un file di configurazione scritto in Ruby che descrive a Vagrant come creare e configurare le VM. Questo singolo file rende il laboratorio portabile, riproducibile e facilmente modificabile

2.1 Analisi del Vagrantfile

Per rendere il codice più pulito e manutenibile, il `Vagrantfile` è stato strutturato in blocchi logici distinti. L'approccio segue il principio **DRY (Don't Repeat Yourself)**, separando la definizione dei dati dalla logica di configurazione.

2.1.1 Blocco 1: definizione dei dati di configurazione

La prima parte del file è dedicata alla centralizzazione di tutti i parametri delle macchine virtuali in una struttura dati Ruby, un Hash chiamato `SERVER`. Questo blocco agisce come un "inventario" del nostro laboratorio: ogni macchina è definita con le sue proprietà specifiche come l'hostname, l'indirizzo IP, la RAM allocata e il playbook Ansible che ne gestirà la configurazione. Questo metodo rende estremamente semplice aggiungere nuove macchine o modificare quelle esistenti, intervenendo solo in questo punto.

```
# definisco le caratteristiche che deve avere ogni macchina virtuale
    come il nome, l'indirizzo ip, la RAM e lo script ANSIBLE associato
SERVER = {
  "lb" => {
    hostname: "loadbalancer",
    ip: "192.168.50.5",
    ram: 1024,
    playbook: "ansible/playbook-lb.yml"
  },
  "web1" => {
    hostname: "webserver1",
    ip: "192.168.50.10",
    ram: 1024,
    playbook: "ansible/playbook-web.yml"
  },
  "web2" => {
    hostname: "webserver2",
    ip: "192.168.50.11",
    ram: 1024,
    playbook: "ansible/playbook-web.yml"
  },
  "db" => {
    hostname: "database",
    ip: "192.168.50.20",
    ram: 2048,
    playbook: "ansible/playbook-db.yml"
  }
}
```

Listing 1: Definizione centralizzata dei parametri delle VM

2.1.2 Blocco 2: ciclo di creazione e configurazione di base

La seconda parte contiene la logica principale. Il comando `Vagrant.configure("2")` inizializza la configurazione. Successivamente, un ciclo `SERVER.each` itera sulla struttura dati definita in precedenza e, per ogni elemento, crea una definizione di VM tramite `config.vm.define`. All'interno di questo blocco vengono impostati i parametri fondamentali della macchina, come l'immagine del sistema operativo da usare (`node.vm.box`), il suo nome di rete (`node.vm.hostname`) e la configurazione della rete privata. Questi comandi sono il cuore pulsante del file, trasformando una semplice lista di dati in istruzioni concrete per l'hypervisor. La scelta della "box" garantisce che ogni macchina parta da uno stato identico e pulito, eliminando le incertezze tipiche dei setup manuali. La definizione della rete privata, in particolare, è un passo architetturale critico, poiché crea il perimetro di sicurezza che isolerà il database.

```
Vagrant.configure("2") do |config|
  #qui, leggo ogni singolo elemento dell'HASH creato in precedenza
  SERVER.each do |server_name, server_config|
    config.vm.define server_name do |node|
      # specifico il sistema operativo da utilizzare
      node.vm.box = "ubuntu/jammy64"

      # specifico un nome a ogni macchina, per poterla riconoscere
      node.vm.hostname = server_config[:hostname]

      # creo una rete privata
      node.vm.network "private_network", ip: server_config[:ip]

    end
  end
end
```

Listing 2: Logica di iterazione e configurazione base delle VM

2.1.3 Blocco 3: configurazione del provider e provisioning

L'ultima parte, interna al ciclo, si occupa di definire "cosa c'è sopra" e "cosa c'è dentro" la macchina virtuale. La distinzione tra provider e provisioner è un concetto chiave in Vagrant, poiché permette di separare la configurazione dell'infrastruttura a livello hardware da quella del software che verrà eseguito al suo interno. Questa separazione rende il processo più ordinato e facile da debuggare, assegnando a ogni blocco una responsabilità chiara e definita.

- **Provider Configuration**: il blocco `node.vm.provider` definisce parametri specifici per l'hypervisor in uso, in questo caso VirtualBox. Qui vengono allocate le risorse hardware come la quantità di RAM (`vb.memory`) e il numero di CPU (`vb.cpus`);
- **Provisioning**: il blocco `node.vm.provision` è il cuore dell'automazione software. Specifica a Vagrant di usare Ansible per configurare la macchina una volta creata. L'opzione `ansible_local` è fondamentale perché istruisce Vagrant a installare Ansible direttamente sulla VM guest per poi eseguire il playbook specificato. Questo approccio è ideale perché non richiede che Ansible sia preinstallato sulla macchina host, garantendo la massima portabilità del progetto.

```
      # qui specifico l'hypervisor, nel mio caso, virtualbox
      node.vm.provider "virtualbox" do |vb| # richiamo i comandi da
    associare a virtualbox con "vb"

        vb.name = server_config[:hostname]
        vb.memory = server_config[:ram]
        vb.cpus = 2
      end

      # lanciamo Ansible per installare tutto il software necessario
      node.vm.provision "ansible_local" do |ansible|
        ansible.install = true
        ansible.playbook = server_config[:playbook]
        ansible.version = "latest"
      end
    end
  end
end
```

Listing 3: configurazione hardware e automazione software

L'esecuzione del comando `vagrant up` interpreta questo file e crea l'intera infrastruttura, come mostrato nell'interfaccia di gestione di VirtualBox in figura 2.

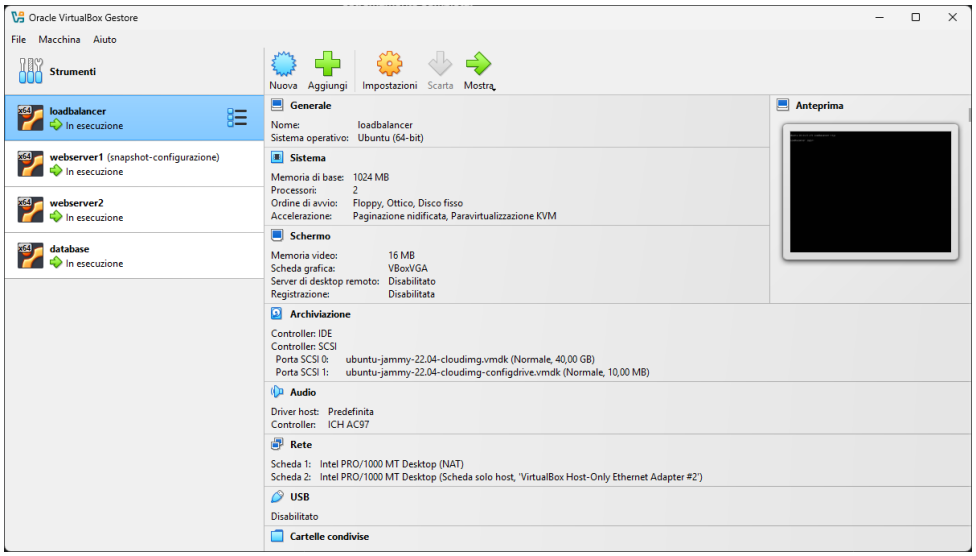

Figura 2: L'interfaccia di Oracle VM VirtualBox che mostra le quattro macchine virtuali create e in esecuzione dopo il comando *vagrant up*

3 Gestione della configurazione con Ansible

Una volta che Vagrant ha creato le VM, **Ansible** subentra per automatizzare la loro configurazione software. Ansible si distingue per la sua architettura **agentless**: non richiede l'installazione di alcun software client ("agente") sulle macchine da gestire. Opera tramite connessioni SSH standard, inviando piccoli moduli Python che eseguono i compiti e poi vengono rimossi. Questo approccio semplifica notevolmente la gestione e riduce il carico sui server.
Ansible opera in modo **dichiarativo**: si descrive lo stato finale desiderato del sistema (es. "Nginx deve essere installato e attivo") in file **YAML** chiamati *playbook*. La scelta di YAML non è casuale: la sua sintassi pulita e basata sull'indentazione lo rende estremamente leggibile anche per chi non è un programmatore. Ansible si occupa poi di raggiungere lo stato descritto, garantendo che server con lo stesso ruolo (es. `web1` e `web2`) siano configurati in modo identico ed eliminando il "configuration drift".

3.1 Struttura e principi dei Playbook

Un concetto chiave in Ansible è l'**idempotenza**. Un'operazione idempotente, se eseguita più volte, produce lo stesso risultato della prima esecuzione. Ad esempio, se un task dichiara che un pacchetto deve essere installato, Ansible lo installerà solo la prima volta; le esecuzioni successive rileveranno che il pacchetto è già presente e non eseguiranno alcuna azione. Questo rende i playbook sicuri da rieseguire e prevedibili nel loro comportamento.
Per massimizzare la riusabilità del codice, le operazioni comuni a tutte le macchine (aggiornamento, installazione di tool di base) sono state raggruppate in un playbook separato, `playbook-common.yml`, e importate dove necessario.

3.1.1 Struttura di un Playbook

Un playbook Ansible è il file centrale che gestisce tutte le operazioni; può essere visto come una vera e propria "ricetta" o un "piano di esecuzione" che Ansible segue dall'inizio alla fine. Sebbene la sua componente principale sia la lista dei task, un playbook può contenere anche altri elementi come la definizione di variabili, l'importazione di ruoli e la dichiarazione di "handler" (task speciali attivati da notifiche), rendendolo uno strumento estremamente versatile. Gli elementi fondamentali sono:

- `hosts`: specifica su quali macchine (o gruppi di macchine) dell'inventario eseguire i task. Nel nostro caso, poiché Ansible viene eseguito localmente su ogni VM, il target è sempre `all` (cioè, la macchina stessa);
- become: yes: istruisce Ansible a eseguire i task con privilegi elevati (equivalente a `sudo`). È necessario per operazioni come l'installazione di software o la modifica di file di sistema;
- `tasks`: è la lista delle operazioni da eseguire. Ogni task ha un nome descrittivo (`name`) e richiama un **modulo** Ansible (`apt`, `copy`, `service`, etc.) con i parametri necessari.

3.2 Playbook comune

Questo playbook funge da base di partenza per tutte le macchine del laboratorio, assicurando che ciascuna di esse soddisfi i requisiti minimi prima di procedere con le configurazioni specifiche del proprio ruolo. Contiene i task fondamentali e comuni, come l'aggiornamento del sistema e l'installazione di pacchetti essenziali, garantendo così uno stato di partenza coerente e omogeneo per l'intera infrastruttura.

```yaml
- name: Aggiorno la cache dei pacchetti e il sistema
  apt:
    update_cache: yes  # apt-get update
    upgrade: dist      # apt-get dist-upgrade

- name: Installo pacchetti di base essenziali
  apt:
    name:
      - build-essential
      - python3-pip
      - git
      - curl
      - vim
    state: present # questo garantisce che i pacchetti siano installati
```

Listing 4: ansible/playbook-common.yml

3.3 Playbook del Load Balancer

Il load balancer è il punto di ingresso della nostra applicazione.
Il suo playbook installa Nginx, un web server leggero e ad alte prestazioni, ideale per questo ruolo. Il cuore della configurazione è il file `lb-nginx.conf`, che definisce la logica di reverse proxy e bilanciamento del carico in modalità *Round Robin* (l'algoritmo di default).

```yaml
- hosts: all
  become: yes
  tasks:
    - name: Importo i task di configurazione comune
      import_tasks: playbook-common.yml

    - name: Installo il web server Nginx
      apt:
        name: nginx
        state: latest

    - name: Copio il file di configurazione per il Load Balancer
      copy:
        src: files/lb-nginx.conf # file sorgente sulla macchina host
        dest: /etc/nginx/sites-available/default # destinazione VM

    - name: Riavvio Nginx per applicare la nuova configurazione
      service:
        name: nginx
        state: restarted #forza il riavvio del servizio
```

Listing 5: ansible/playbook-lb.yml per il Load Balancer

3.4 Playbook dei Web Server

Questo playbook, essendo identico per `web1` e `web2`, garantisce la coerenza tra i server applicativi. Oltre a installare Nginx, introduce un task di manutenzione automatizzata tramite il modulo `cron` di Ansible. Questo modulo permette di gestire i cron job in modo idempotente, assicurando che l'operazione di backup sia schedulata correttamente.

```yaml
- hosts: all
  become: yes
  tasks:
    - name: Importo i task di configurazione comune
      import_tasks: playbook-common.yml

    - name: Installo il web server Nginx
      apt:
        name: nginx
        state: latest

    - name: Pianifico un backup giornaliero dei file del sito web
      cron:
        name: "Backup giornaliero sito web"
        minute: "0"
        hour: "4"
        job: "tar -czf /var/backups/sito-web-$(date +\\%Y-\\%m-\\%d).tar
    .gz /var/www/html"
```

Listing 6: ansible/playbook-web.yml per i Web Server

3.5 Configurazione del Reverse Proxy (Load Balancer)

Il cuore del load balancer è il suo file di configurazione Nginx. La direttiva 'upstream' definisce un gruppo di server a cui inoltrare il traffico, mentre il blocco 'location' specifica che tutte le richieste in entrata debbano essere passate a questo gruppo tramite '*proxy_pass*'. Le intestazioni '*proxy_set_header*' sono fondamentali per inoltrare ai server web le informazioni originali del client.

```nginx
upstream web_servers {
    server 192.168.50.10; # primo web server
    server 192.168.50.11; # secondo web server
}

server {
    listen 80;
    server_name _; #risponde a qualsiasi richiesta

    location / {
        proxy_pass http://web_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Listing 7: files/lb-nginx.conf: configurazione Nginx per il reverse proxy

3.6 Playbook del Database

Il server di database è un componente critico e va trattato con particolare attenzione alla sicurezza. Il suo playbook installa MariaDB, un fork open source di MySQL molto diffuso. Sebbene il playbook si limiti all'installazione, una configurazione di produzione dovrebbe includere task di *hardening* per la messa in sicurezza del servizio.

```yaml
- hosts: all
  become: yes
  tasks:
    - name: Importo i task di configurazione comune
      import_tasks: playbook-common.yml

    - name: Installo MariaDB Server e client
      apt:
        name:
          - mariadb-server
          - mariadb-client
        state: latest
```

Listing 8: ansible/playbook-db.yml per il database server

3.7 Gestione dei dati sensibili: Ansible Vault

Nei playbook presentati, non sono stati gestiti dati sensibili come password o chiavi API per semplicità. Tuttavia, in un ambiente di produzione, è fondamentale evitare di scrivere informazioni critiche in chiaro nei file di configurazione. Per questo tipo di esigenza, Ansible offre una soluzione integrata e sicura: **Ansible Vault**.
Ansible Vault è una funzionalità che permette di criptare singoli file o variabili all'interno del progetto. Un file criptato con Vault può essere tranquillamente incluso nel sistema di controllo versione (come Git), poiché il suo contenuto è illeggibile senza la password corretta. Durante l'esecuzione del playbook, è possibile fornire la password ad Ansible, che decripterà i dati al volo per utilizzarli nei task. Ad esempio, il playbook del database potrebbe essere esteso per impostare una password di root per MariaDB, leggendola da un file protetto da Vault, garantendo così la massima sicurezza e aderenza alle best practice.

4 Procedure di gestione e manutenzione

Un laboratorio, per quanto automatizzato, richiede procedure definite per la manutenzione ordinaria e per la gestione di imprevisti (disaster recovery). L'affidabilità di un sistema non dipende solo dalla sua architettura, ma anche dalla robustezza delle procedure operative che ne garantiscono la stabilità e la recuperabilità nel tempo.

4.1 Gestione degli snapshot

Gli snapshot di VirtualBox, gestibili comodamente tramite Vagrant, creano punti di ripristino istantanei dello stato di una VM, includendo la memoria RAM, le impostazioni e lo stato del disco. Sono lo strumento ideale prima di effettuare modifiche potenzialmente rischiose, come un aggiornamento del software o un cambio di configurazione, poiché consentono di annullare le operazioni e tornare allo stato precedente in pochi secondi.

- **Creazione di uno snapshot:** `vagrant snapshot save <vm-name> <snapshot-name>`
- **Ripristino di uno snapshot:** `vagrant snapshot restore <vm-name> <snapshot-name>`

4.1.1 Snapshot vs. backup

È fondamentale comprendere che uno snapshot **non è un backup**. Uno snapshot è un disco differenziale che registra solo le modifiche rispetto allo stato precedente; dipende quindi dal disco originale e da eventuali snapshot precedenti. Se il disco base venisse cancellato o si corrompesse, lo snapshot diventerebbe inutilizzabile. Pertanto, gli snapshot sono perfetti per ripristini a brevissimo termine, ma non devono mai essere considerati una strategia di backup affidabile per il disaster recovery.

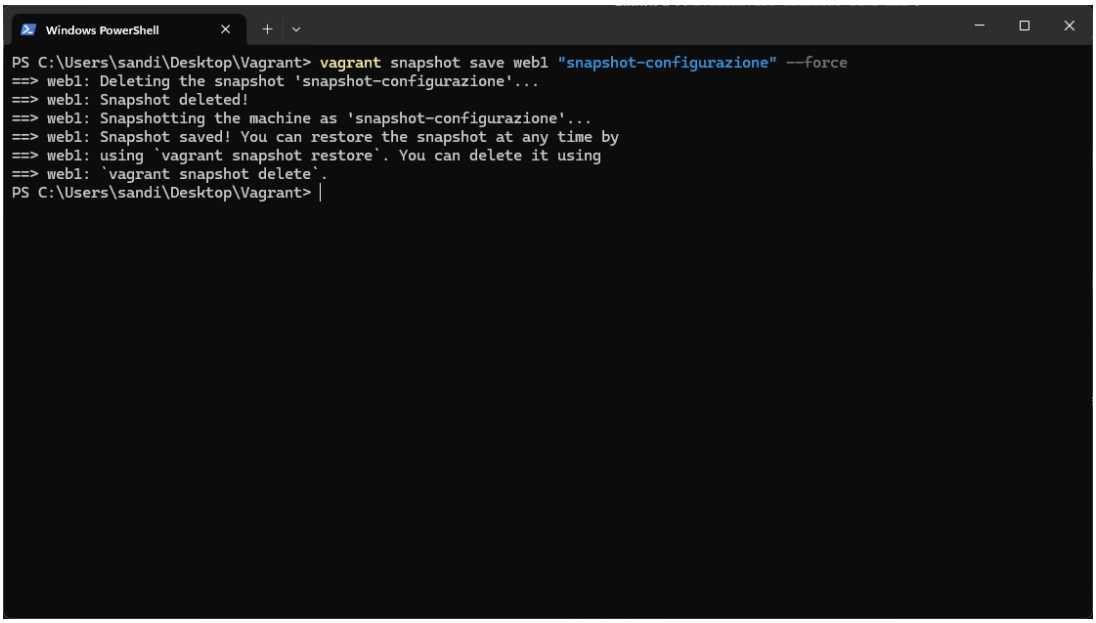

Figura 3: Comando per la creazione di uno snapshot della VM 'webserver1'

4.2 Backup completo e migrazione (Disaster Recovery)

Per un vero disaster recovery o per la migrazione dell'infrastruttura, è necessario un backup completo, auto-contenuto e portabile. A questo scopo è stato sviluppato lo script Batch `backup.bat`, che orchestra l'utility a riga di comando di VirtualBox, `VBoxManage.exe`, per esportare una VM in formato **OVA (Open Virtualization Appliance)**. Questo formato standard, di fatto un archivio TAR, impacchetta in un unico file sia i dischi virtuali (in formato VMDK) sia la descrizione hardware della VM (in un file OVF basato su XML). Questa natura auto-contenuta lo rende agnostico rispetto all'hypervisor e ideale per l'archiviazione a lungo termine. L'automazione tramite script, inoltre, garantisce che la procedura di backup sia eseguita sempre in modo identico e affidabile, eliminando il rischio di errori umani tipico delle operazioni manuali.

4.2.1 Blocco 1: inizializzazione

La prima parte dello script imposta le variabili fondamentali, come il nome della VM da backuppare e i percorsi. Una pratica importante qui adottata è la generazione dinamica del nome del file di backup, che include la data corrente per creare archivi unici e facilmente identificabili. Subito dopo, implementa un meccanismo di "locking": crea un file temporaneo `backup.lock` che impedisce l'avvio di un'altra istanza dello script mentre una è già in esecuzione. Questo semplice controllo previene sovrapposizioni e possibili corruzioni dei dati durante il processo di esportazione.

```
@echo off
rem impostazioni
set VM_NAME=webserver1
set BACKUP_PATH=C:\Backups
set BACKUP_NAME=%BACKUP_PATH%\%VM_NAME%_backup_%date:~-4%%date:~4,2%%
    date:~-10,2%.ova
set VBOXMANAGE="C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
set LOCKFILE=backup.lock

rem controllo lock
if not exist %BACKUP_PATH% mkdir %BACKUP_PATH%
if exist %LOCKFILE% (
    echo Script gia' in esecuzione. Uscita...
    exit /b
)
echo > %LOCKFILE%
```

Listing 9: impostazione delle variabili e meccanismo di lock

4.2.2 Blocco 2: gestione dello stato della VM

Questa è la sezione più delicata dello script. Prima di avviare il backup, controlla se la macchina virtuale è in esecuzione, in caso affermativo, lo script non spegne brutalmente la VM, ma invia un segnale di spegnimento controllato (ACPI), che simula la pressione del tasto di accensione. Lo script attende poi in un ciclo che la VM completi lo shutdown. Questo garantisce che il sistema operativo guest si arresti correttamente, chiudendo file e servizi, per assicurare la massima consistenza dei dati nel backup.

```
echo Controllo lo stato della VM '%VM_NAME%'...
rem controlla se la VM e in stato "running"
%VBOXMANAGE% showvminfo %VM_NAME% --machinereadable | findstr "VMState
    =\"running\"" >nul
if %errorlevel%==0 (
    echo La VM e' accesa. Spegnimento controllato in corso...
    set VM_WAS_RUNNING=1
    %VBOXMANAGE% controlvm %VM_NAME% acpipowerbutton

    rem attende in un ciclo che la VM sia spenta prima di continuare
    :wait_shutdown
    timeout /t 5 /nobreak >nul
    %VBOXMANAGE% showvminfo %VM_NAME% --machinereadable | findstr "
    VMState=\"poweroff\"" >nul
    if errorlevel 1 goto wait_shutdown
    echo VM spenta.
) else (
    echo La VM e' gia' spenta.
    set VM_WAS_RUNNING=0
)
```

Listing 10: spegnimento controllato della VM

4.2.3 Blocco 3: esportazione, riavvio e pulizia

Una volta che la VM è spenta, lo script esegue il comando `VBoxManage export` per creare il file `.ova`. Al termine del backup, se la macchina era stata spenta appositamente, viene riavviata in modalità *headless* (senza interfaccia grafica) per ripristinare lo stato operativo. Infine, il file di lock viene rimosso, permettendo future esecuzioni dello script.

```
echo Avvio esportazione della VM in: %BACKUP_NAME%
%VBOXMANAGE% export %VM_NAME% --output %BACKUP_NAME%
if %errorlevel% neq 0 (
    echo ERRORE durante il backup.
    goto cleanup
)
echo Backup completato con successo!

rem se la VM era accesa all'inizio, la riavviamo
if %VM_WAS_RUNNING%==1 (
    echo Riavvio della VM in corso...
    %VBOXMANAGE% startvm %VM_NAME% --type headless
)

rem rimuove il file di lock e termina
:cleanup
del %LOCKFILE%
echo.
pause
```

Listing 11: esportazione, riavvio e pulizia finale

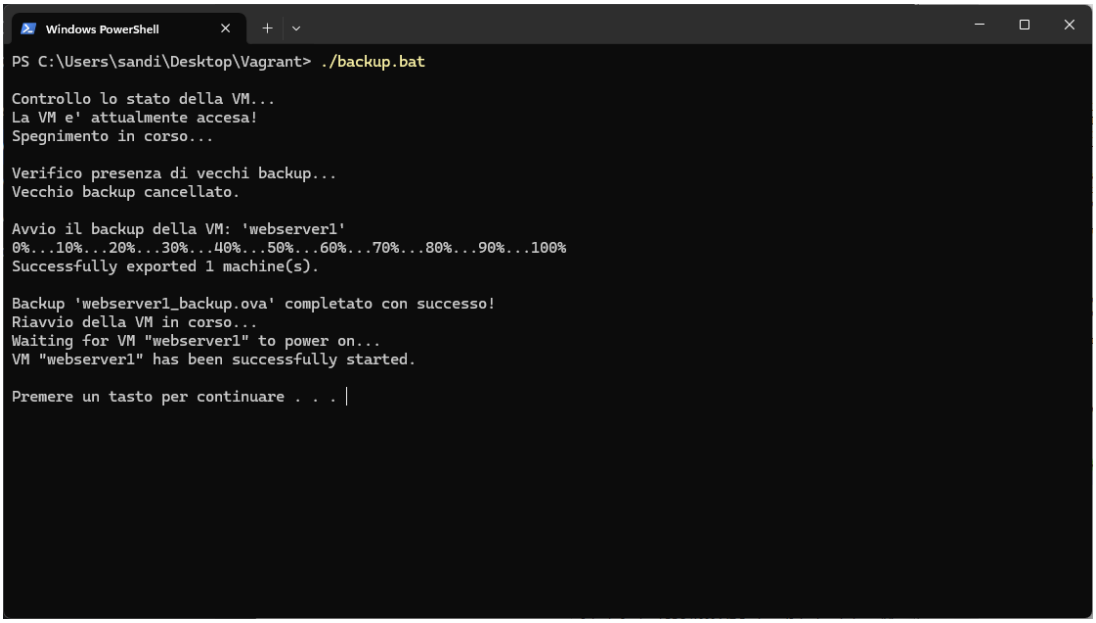

Figura 4: esecuzione dello script ‘backup.bat‘ che mostra lo spegnimento, l’esportazione e il riavvio

5 Verifica funzionale e test di resilienza

Il collaudo finale è un passaggio cruciale non solo per validare l'architettura, ma per dimostrare empiricamente che i principi di progettazione adottati si traducono in un sistema funzionante, robusto e affidabile. I test sono stati suddivisi in due scenari principali: il primo verifica il corretto funzionamento del sistema a regime (bilanciamento del carico), mentre il secondo ne simula un guasto per testarne la capacità di resistere e recuperare (alta affidabilità e failover).

5.1 Test del Load Balancing

Il primo test ha lo scopo di verificare che il load balancer distribuisca correttamente il traffico tra i due server web del backend. Per rendere visibile questo meccanismo, che altrimenti sarebbe trasparente per l'utente, le pagine di benvenuto di Nginx su ciascun web server sono state personalizzate.
Accedendo tramite browser all'indirizzo IP del load balancer (`http://192.168.50.5`), e ricaricando la pagina più volte, si osserva l'alternarsi dei messaggi "Pagina servita da WEB-1" e "Pagina servita da WEB-2". Questo conferma che Nginx sta applicando correttamente l'algoritmo di bilanciamento di default, il **Round Robin**, che consiste nell'inoltrare ogni nuova richiesta al server successivo nella lista, in modo ciclico.

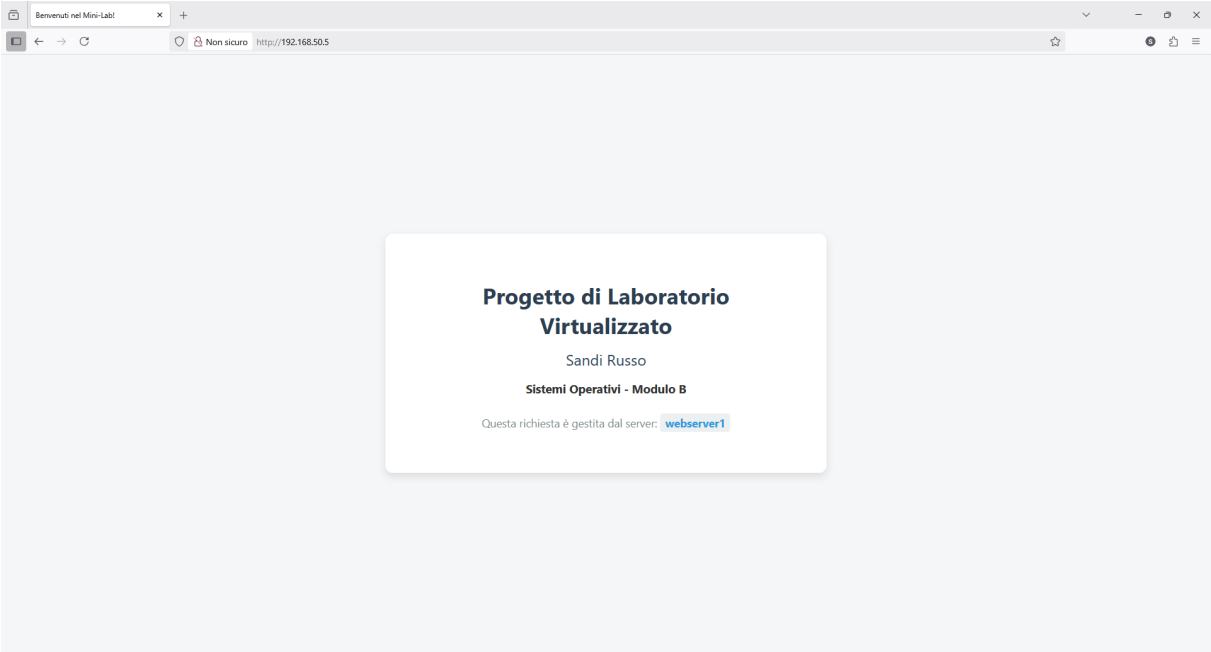

Figura 5: risposta dal primo web server, accessibile tramite l'IP del load balancer

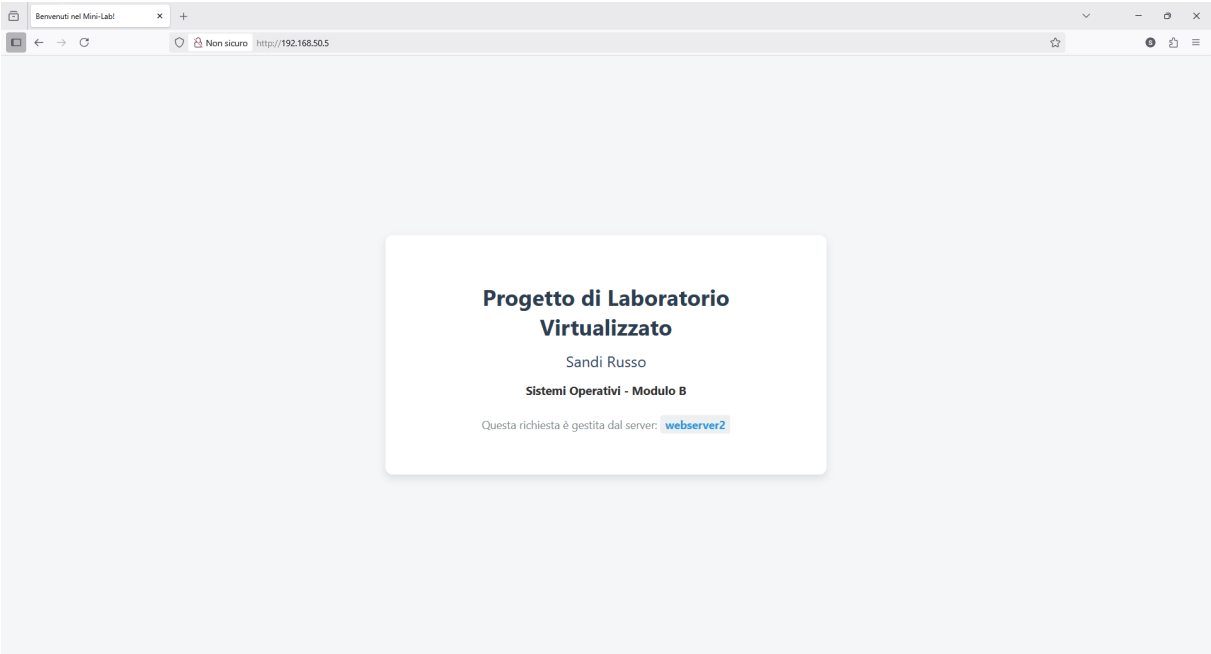

Figura 6: dopo un refresh, il load balancer inoltra la richiesta al secondo web server

5.2 Test di alta affidabilità (Failover)

Questo test simula un guasto critico per verificare che l'architettura sia realmente ad alta affidabilità. Uno dei due web server (`web1`) è stato spento forzatamente tramite Vagrant.

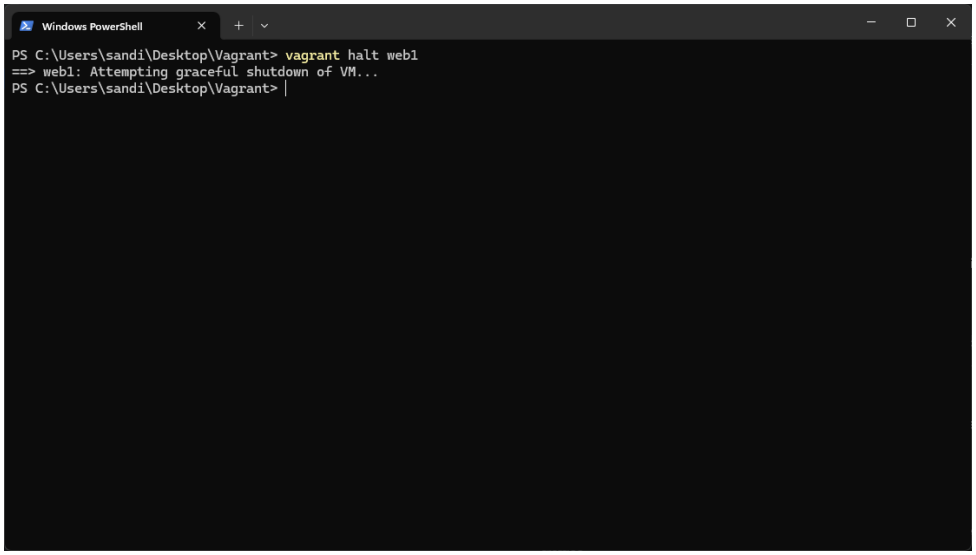

Figura 7: simulazione del guasto di un web server tramite il comando '*vagrant halt*'

Ricaricando la pagina del browser, il servizio è risultato ancora perfettamente accessibile, mostrando costantemente la pagina servita da WEB-2. Questo dimostra il corretto funzionamento del **failover**.
Questo meccanismo di health check passivo è fondamentale per garantire la continuità del servizio (business continuity) anche in presenza di guasti.

```
vagrant@webserver2: ~
PS C:\Users\sandi\Desktop\Vagrant> vagrant ssh web2
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-151-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Aug 12 20:52:54 UTC 2025

  System load:             0.05
  Usage of /:              7.2% of 38.70GB
  Memory usage:            21%
  Swap usage:              0%
  Processes:               106
  Users logged in:         0
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd17:625c:f037:2:29:1eff:fe73:1fd5


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Tue Aug 12 20:52:55 2025 from 10.0.2.2
vagrant@webserver2:~$ sudo tail /var/log/nginx/access.log
192.168.50.5 - - [05/Aug/2025:21:17:07 +0000] "GET / HTTP/1.0" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.50.1 - - [05/Aug/2025:21:17:22 +0000] "GET / HTTP/1.1" 200 396 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.50.1 - - [05/Aug/2025:21:17:22 +0000] "GET /favicon.ico HTTP/1.1" 404 197 "http://192.168.50.11/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safa
ri/537.36"
192.168.50.5 - - [12/Aug/2025:20:30:55 +0000] "GET /favicon.ico HTTP/1.0" 404 564 "http://192.168.50.5/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safar
i/537.36"
192.168.50.5 - - [12/Aug/2025:20:40:45 +0000] "GET /favicon.ico HTTP/1.0" 404 564 "http://192.168.50.5/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safar
i/537.36"
192.168.50.5 - - [12/Aug/2025:20:40:46 +0000] "GET /favicon.ico HTTP/1.0" 404 564 "http://192.168.50.5/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safar
i/537.36"
192.168.50.5 - - [12/Aug/2025:20:44:18 +0000] "GET /favicon.ico HTTP/1.0" 404 162 "http://192.168.50.5/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:141.0) Gecko/20100101 Firefox/141.0"
192.168.50.5 - - [12/Aug/2025:20:46:11 +0000] "GET / HTTP/1.0" 200 1614 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:141.0) Gecko/20100101 Firefox/141.0"
192.168.50.5 - - [12/Aug/2025:20:46:11 +0000] "GET /favicon.ico HTTP/1.0" 404 162 "http://192.168.50.5/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:141.0) Gecko/20100101 Firefox/141.0"
192.168.50.5 - - [12/Aug/2025:20:51:54 +0000] "GET / HTTP/1.0" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:141.0) Gecko/20100101 Firefox/141.0"
vagrant@webserver2:~$
```

Figura 8: analisi dei log di accesso su ‘web2‘ dopo lo spegnimento di ‘web1‘, che mostra l’arrivo di tutte le nuove richieste

5.2.1 Test di ripristino (Recovery)

A completamento del test di failover, è stata verificata la capacità del sistema di ripristinarsi automaticamente. La macchina `web1` è stata riavviata con il comando `vagrant up web1`. Dopo pochi istanti, ricaricando nuovamente la pagina del browser, si è potuto osservare che il load balancer ha ricominciato ad alternare le risposte tra WEB-1 e WEB-2. Questo conferma che Nginx ha rilevato il ritorno online del server e lo ha reintegrato automaticamente nel pool di bilanciamento, dimostrando la capacità di *self-healing* dell’infrastruttura.

6 Conclusioni e sviluppi futuri

Il progetto ha permesso di raggiungere con successo tutti gli obiettivi prefissati. È stato realizzato un laboratorio virtualizzato che non è un semplice insieme di macchine, ma un sistema complesso, resiliente e gestito con metodologie professionali. L'uso combinato di **Vagrant** e **Ansible** ha permesso di applicare concretamente il paradigma dell'Infrastructure as Code, creando un'infrastruttura completamente definita da codice, automatizzata e facilmente riproducibile.
Le procedure di manutenzione, come la gestione degli snapshot e i backup completi tramite script, insieme ai test di resilienza, hanno confermato la robustezza della soluzione, dimostrando le potenzialità delle pratiche DevOps nell'amministrazione di sistema moderna.

6.1 Possibili Sviluppi Futuri

Il laboratorio attuale costituisce una solida base per future esplorazioni e miglioramenti. Tra le possibili direzioni:

- **Containerizzazione:** Sostituire le VM con container **Docker** e orchestrarli con **Docker Compose** o **Kubernetes**. Questo porterebbe a un utilizzo ancora più efficiente delle risorse e a tempi di avvio quasi istantanei.
- **Integrazione Continua / Distribuzione Continua (CI/CD):** Implementare una pipeline CI/CD (es. con GitLab CI o Jenkins) che, a ogni modifica del codice dell'applicazione o dell'infrastruttura, esegua automaticamente test e deployment, automatizzando ulteriormente il ciclo di vita del software.
- **Monitoring e Logging Centralizzato:** Integrare strumenti come **Prometheus** e **Grafana** per il monitoraggio delle performance in tempo reale e uno stack **ELK** (Elasticsearch, Logstash, Kibana) per aggregare e analizzare i log di tutte le macchine in un'unica dashboard.

Questi passi evolutivi permetterebbero di trasformare il laboratorio in una piattaforma ancora più sofisticata e allineata con gli standard più avanzati dell'industria tecnologica.